Few-Shot Learning

Learning from Limited Examples

Sneha S A

Guided by Prof. Rejimol Robinson R R

Contents

1. What is Few-Shot Learning?
2. Why is it Needed?
3. Why is it Difficult?
4. N-Way K-Shot, Support & Query Sets
5. Prototypical Network – How Classification Works
6. Numerical Example
7. Distance Measures
8. Architecture & Training, Episodic Training
9. Major FSL Approaches
10. Prior Knowledge Pyramid
11. Comparison of the Three Research Papers
12. ALP-Net and SSL-ALP-Net (Medical Segmentation)
13. Applications of FSL
14. FSL in Our Android Malware Detection Project
15. Evaluation, Conclusion

What is Few-Shot Learning?

Definition: Few-Shot Learning (FSL) is a machine learning setting where a model learns to recognize new classes using only a very small number of labeled examples per class, instead of thousands.

Conventional Learning vs FSL

- Conventional: 1 class needs 1000s of labeled images $\rightarrow$ learn from scratch
- FSL: 1 class needs 1–10 labeled images $\rightarrow$ reuse prior knowledge

$$\textbf{Prior Knowledge} + \textbf{Few Labeled Examples} \longrightarrow \textbf{Generalization}$$

One-shot / Few-shot intuition

- One-shot: exactly 1 example per class ($K = 1$)
- Few-shot: a handful of examples per class ($K = 2$ to ~ 20)
- Zero-shot: no examples at all, only a text/attribute description

Why is Few-Shot Learning Needed?

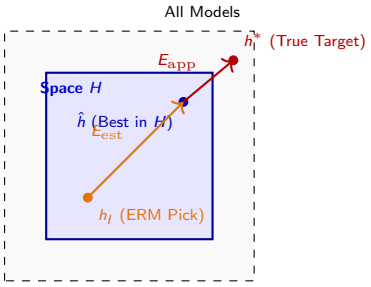
- **Data scarcity** – many real classes never have large labeled datasets
- **Expensive labelling** – expert annotation (radiologists, malware analysts) is costly
- **Rare / new classes** – new categories keep appearing after deployment

Real-world examples:

- Medical imaging – rare disease/tumor types with only a few scans
- Malware – new malware families with very few labeled samples
- NLP – new intents/languages with limited annotated text
- Recommendation – new users/items with almost no history (cold start)

Why conventional supervised learning struggles: it assumes large labeled data is always available to estimate class boundaries accurately – this assumption breaks in all scenarios above.

Why is Few-Shot Learning Difficult?



Model Notation:

- h: Any hypothesis/model in space H.
- h^*: The ideal target model (true risk minimizer).
- $\hat{h}$: The best possible model inside H.
- h_I: Model chosen by ERM using I training samples.

Generalization Error Decomposition:

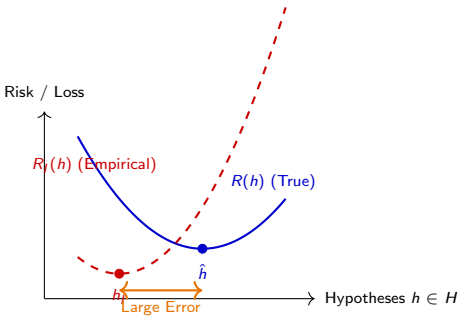
$$R(h_I) - R(h^*) = \underbrace{R(\hat{h}) - R(h^*)}_{E_{\text{app}}(H)} + \underbrace{R(h_I) - R(\hat{h})}_{E_{\text{est}}(H,I)}$$

- $E_{\text{app}}(H)$ (**Approximation Error**): Caused by H capacity; independent of sample size I.
- $E_{\text{est}}(H, I)$ (**Estimation Error**): Depends on sample size I.

FSL Bottleneck: When I is very small, $E_{\text{est}}(H, I)$ explodes, causing ERM (h_I) to generalize poorly.

Source: Wang et al., 2020, Sec. 2.3.1–2.3.2

Why Does ERM Become Unreliable in Few-Shot Learning?



The ERM Failure Mechanism

Empirical Risk Minimization (ERM) assumes $R_I(h) \approx R(h)$.

- When sample count I is large, $R_I(h)$ converges to true risk $R(h)$, bringing $h_I \to \hat{h}$.
- In **Few-Shot Learning** (I is small), empirical risk $R_I(h)$ fluctuates severely away from $R(h)$.
- Minimizing $R_I(h)$ selects an h_I that overfits small sample noise and strays far from optimal $\hat{h}$.

$$\text{Small } I \implies \text{Unreliable } R_I(h) \implies h_I \not\approx \hat{h} \text{ (Severe Overfitting)}$$

Source: Wang et al., 2020, Sec. 2.3.1–2.3.2

N-Way K-Shot, Support and Query Sets

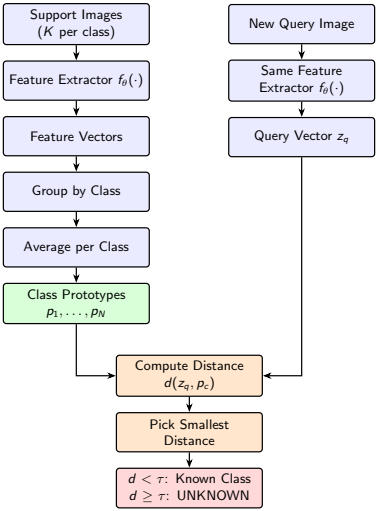
- **N-way** = number of classes in the task
- **K-shot** = number of labeled examples given per class
- **Support set (S)** = the small labeled set given for the new task
- **Query set (Q)** = unlabeled examples the model must classify

Example: 3-way 2-shot task

Class	Support Set (labeled)	Query Set (to classify)
A	img1, img2	?
B	img1, img2	?
C	img1, img2	?

3 classes $\times$ 2 examples each = 6 support images total. The model classifies each query image using *only* these 6 references.

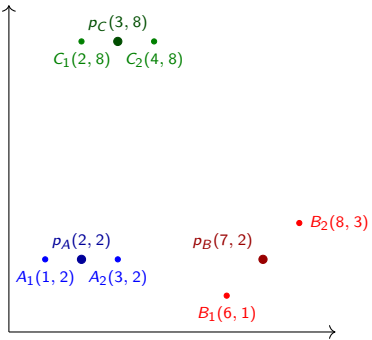
How Few-Shot Classification Works: Prototypical Network



Source: Architecture based on Snell et al. (Prototypical Networks), as reviewed in Wang et al., 2020, Fig. 7

Prototypical Classification: Numerical Example (Part 1 – Setup)

Setup: 3-way, 2-shot task. To make this concrete, assume the feature extractor f_θ maps every image to a simple 2D embedding (in practice this vector has hundreds of dimensions, but 2D lets us visualize it exactly the same way).



Support embeddings given:

Class A: $f(x_{A1}) = (1,2),\ f(x_{A2}) = (3,2)$

Class B: $f(x_{B1}) = (6,1),\ f(x_{B2}) = (8,3)$

Class C: $f(x_{C1}) = (2,8),\ f(x_{C2}) = (4,8)$

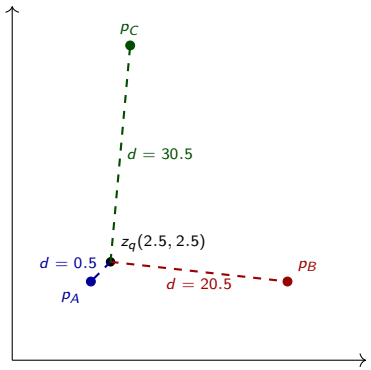
Step 1 – Compute each prototype:

$$p_A = \frac{(1,2)+(3,2)}{2} = (\mathbf{2},\mathbf{2})$$

$$p_B = \frac{(6,1)+(8,3)}{2} = (\mathbf{7},\mathbf{2})$$

$$p_C = \frac{(2,8)+(4,8)}{2} = (\mathbf{3},\mathbf{8})$$

Prototypical Classification: Numerical Example (Part 2 – Decision)



Query embedding: $z_q = f_\theta(x_{\text{query}}) = (2.5, 2.5)$

Step 2 – Squared Euclidean distance:

$$d(z_q, p_A) = (2.5-2)^2 + (2.5-2)^2 = \mathbf{0.5}$$

$$d(z_q, p_B) = (2.5-7)^2 + (2.5-2)^2 = \mathbf{20.5}$$

$$d(z_q, p_C) = (2.5-3)^2 + (2.5-8)^2 = \mathbf{30.5}$$

Step 3 – Decision rule:

$$\hat{y} = \arg\min_c \; d(z_q, p_c) = \mathbf{A} \quad \text{(min: 0.5)}$$

Why A wins: z_q sits right next to class A's prototype (p_A), yielding the smallest distance ($d = 0.5$) on the plot.

Distance and Similarity Measures

Euclidean Distance

$$d(z_q, p_c) = \|z_q - p_c\|^2 = \sum_i (z_{q,i} - p_{c,i})^2$$

Measures the straight-line distance between two points in feature space. Sensitive to both the *direction* and the *magnitude* of the vectors – used by default in Prototypical Networks.

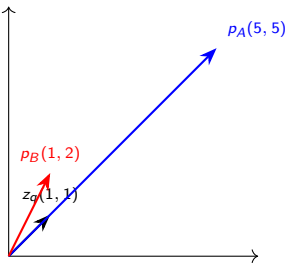
Cosine Similarity

$$\text{sim}(z_q, p_c) = \frac{z_q \cdot p_c}{\|z_q\| \, \|p_c\|} = \cos(\phi)$$

Measures only the *angle* ϕ between two vectors, ignoring their length – used in Matching Networks and NLP embeddings, where embedding magnitude often carries no useful meaning.

Rule of thumb: Euclidean distance for averaged prototypes (Prototypical Networks); cosine similarity for direct high-dimensional embedding comparison where scale should not matter.

Euclidean vs. Cosine: A Worked Example Where They Disagree



Query $z_q = (1, 1)$. Two candidate prototypes: $p_A = (5, 5)$, $p_B = (1, 2)$.

By Euclidean distance:

$$d(z_q, p_A) = \sqrt{4^2 + 4^2} = 5.66, \quad d(z_q, p_B) = \sqrt{0^2 + 1^2} = 1.0$$

$\Rightarrow$ nearest is p_B (smaller distance)

By cosine similarity:

$$\text{sim}(z_q, p_A) = \frac{5+5}{\sqrt{2}\sqrt{50}} = 1.00, \quad \text{sim}(z_q, p_B) = \frac{1+2}{\sqrt{2}\sqrt{5}} = 0.949$$

$\Rightarrow$ nearest is p_A (higher similarity)

Interpretation: p_A points in *exactly* the same direction as z_q (just scaled up) – cosine similarity says they are identical in direction. Euclidean distance instead penalizes p_A for being far away in magnitude, and prefers the nearby but slightly differently-angled p_B. **The metric you choose can change the predicted class.**

Architecture, Training & Episodic Training

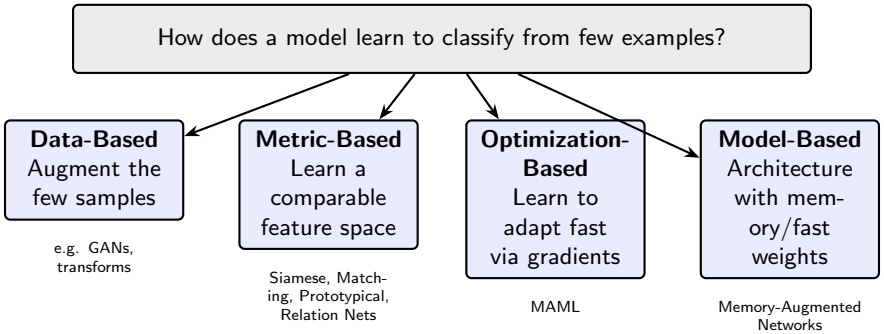
Feature Extractor: a CNN/Transformer f_θ mapping any input to a fixed-length embedding – the SAME network is used for support and query images (shared weights).

Training via Episodes (not normal batches):

1. Sample N classes from the training data
2. Sample K support + Q query examples per class
3. Compute prototypes from the support set
4. Classify the query set using distance to prototypes
5. Compute loss and backpropagate

Why episodic training matters: each episode artificially recreates a mini few-shot task, so the model repeatedly *practices* the exact procedure it will perform at test time on genuinely new classes.

Major Few-Shot Learning Approaches



Data-Based Methods

Core idea: increase the effective amount of training data available for the few-shot task, so ordinary learning becomes more reliable.

- **Data Expansion:** flipping, cropping, rotation, GAN-generated samples
- **Feature Augmentation:** borrow mean/variance statistics from a similar, data-rich "base class" to correct a data-poor "novel class"
- **Semi-supervised use:** bring in unlabeled data from the same domain

This approach works at the **Data Level** – it doesn't change the model or algorithm, only the amount/diversity of information seen.

Source: Song et al., 2023, Sec. 3

Metric-Based Methods

Core idea: learn an embedding function so classification reduces to a simple distance/similarity comparison – no retraining needed per task.

- **Siamese Networks** – compares *pairs* of images, learns same/different
- **Matching Networks** – attention-weighted vote over all support labels
- **Prototypical Networks** – compares query to the mean embedding (prototype) of each class
- **Relation Networks** – *learns* the similarity function itself via a small neural network instead of a fixed formula

All four are variations on the same theme shown earlier – only the comparison mechanism changes.

Source: Wang et al., 2020, Sec. 4.2; Song et al., 2023, Table 9

Optimization-Based Methods: MAML

Core Idea: Learn an initial parameter set θ that adapts to any new task T_i in just a few gradient steps.

1. Inner Loop (Task-Specific Adaptation)

$$\theta_i' = \theta - \alpha \nabla_\theta L_{T_i}(f_\theta)$$

- **Meaning:** Starts at shared base weights θ, evaluates support set loss for task T_i, and takes a quick task step (α) to compute temporary parameters θ_i'.
- **Role:** θ_i' is used only for evaluating Task i.

2. Outer Loop (Meta-Learning the Initialization)

$$\theta \leftarrow \theta - \beta \nabla_\theta \sum_i L_{T_i}(f_{\theta_i'})$$

- **Meaning:** Evaluates performance of temporary parameters θ_i' on query sets across tasks, computing cross-task loss to update base initialization θ (β).
- **Goal:** Shift θ to a sweet spot where taking 1–2 steps on new data achieves high accuracy.

Model-Agnostic: Works seamlessly across architectures (CNNs, ViTs) and domains (classification, RL).

Source: Finn et al., as reviewed in Wang et al., 2020, Fig. 14

Model-Based Methods

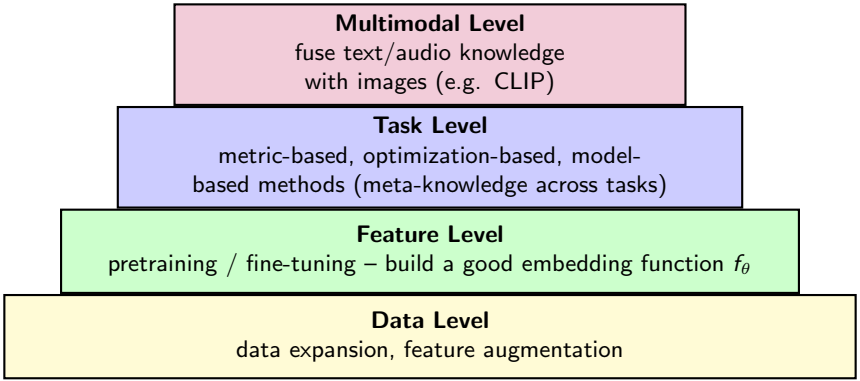
Core idea: build the network architecture itself to rapidly adapt using the support set directly – no gradient-descent adaptation phase needed.

- **Memory-Augmented Networks:** an external key-value memory stores each support example as it is seen; the query is classified by reading the most similar memory entries
- **Fast-weight networks:** a small "meta-learner" network generates temporary, task-specific weights directly from the support set in one shot – no iterative gradient steps needed at test time

This family trades architectural complexity for extremely fast adaptation.

Source: Song et al., 2023, Sec. 5.1

Prior Knowledge Pyramid



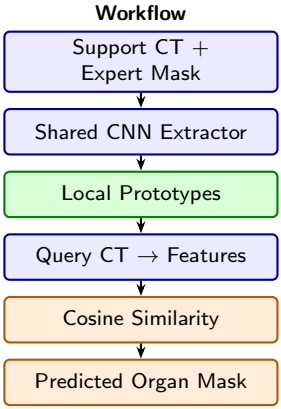
Each level is built on top of the one below it: task-level meta-learning needs a good feature extractor first, which itself needs data to begin with.

Source: Song et al., 2023, Fig. 7, Sec. 2.6

Comparative Analysis

Aspect	Wang et al. (2020)	Song et al. (2023)	Dissanayake et al. (2025)
What the paper investigates	Why learning from very few examples is difficult and how prior knowledge can help FSL.	How FSL has evolved, what strategies are being used, and where prior knowledge is introduced.	How FSL methods perform in **medical image segmentation** and what is required to adapt them to this domain.
Key Insight	With few samples, empirical risk minimization becomes unreliable because the model has insufficient evidence to identify the correct hypothesis.	FSL can exploit prior knowledge at multiple levels: **data, features, tasks, and multimodal information**.	Methods successful on natural images may not transfer directly to medical images because of the **domain gap**.
Main Finding	Prior knowledge reduces the effect of limited data by augmenting information, constraining the hypothesis space, or guiding the learning process.	The field has moved beyond simple fine-tuning toward **metric learning, meta-learning, self-supervised learning, and multimodal/foundation-model approaches**.	The review's comparative experiments show that **self-supervised learning can substantially improve few-shot medical segmentation**; the paper reports stronger gains for SSL-ALP-Net than the corresponding baseline.

ALP-Net: Adaptive Local Prototype Network



Core idea: few-shot organ segmentation from a small labelled support set, using several **local prototypes** instead of one global prototype.

Steps

1. Give a support CT image and its expert mask.
2. A shared CNN extracts features from support and query images.
3. The mask selects organ features; local regions give local prototypes.
4. Each query pixel is compared with the prototypes by cosine similarity.
5. The pixel-wise results form the predicted mask.

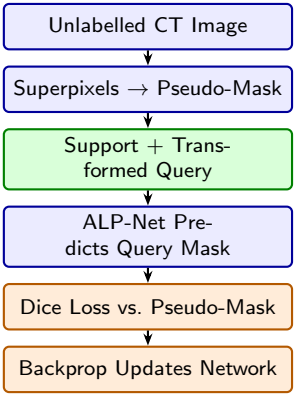
$$\mathrm{sim}(z, p) = \frac{z \cdot p}{\|z\| \, \|p\|}$$

Why local? They keep appearance variation inside an organ that one global average would lose.

Source: Dissanayake et al. (2025).

SSL-ALP-Net: Self-Supervised ALP-Net

Self-supervised training workflow



Core idea: adds a self-supervised training stage to ALP-Net, using unlabelled images and automatic pseudo-masks instead of real organ labels.

Training steps

1. Start with an unlabelled CT image.
2. Superpixels group similar pixels into regions, giving a pseudo-mask.
3. Transform image and mask consistently to form a support-query episode.
4. ALP-Net predicts the mask of the transformed query.
5. Dice loss against the transformed pseudo-mask; backpropagate.

Note: superpixels do not identify the kidney. At test time a labelled support image and kidney mask still define the target organ.

Source: Dissanayake et al. (2025).

ALP-Net vs. SSL-ALP-Net: Results (1-way, 1-shot)

Dice score (%) – abdominal CT organs

Target	ALP-Net	SSL-ALP-Net
Left Kidney	30.91	**63.34**
Right Kidney	27.21	**52.11**
Spleen	40.31	**60.99**
Liver	55.01	**74.21**
Kidney Tumor (KiTS)	**30.45**	23.94

Source: Dissanayake et al., 2025, Sec. 4.1–4.2, Table 6

Observations

- SSL training gives roughly a **20–30 point Dice gain** on organs, with the *same architecture*.
- The gain comes from the **training strategy**, not from a new network design.
- On **tumors**, SSL-ALP-Net is worse than ALP-Net: tumors are small, irregular regions that do not align well with superpixels.

Takeaway: self-supervision helps well-defined organs, but fine-grained lesions remain an open problem.

Applications of Few-Shot Learning

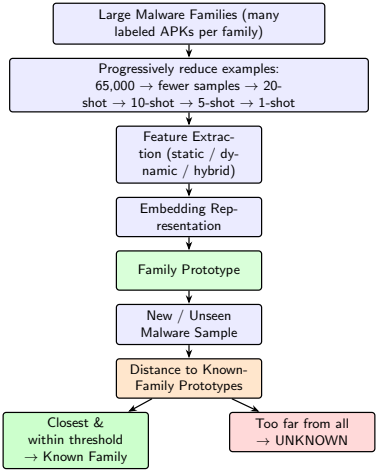
- **Computer Vision** – image classification, object detection, segmentation
- **NLP** – intent classification, low-resource language tasks
- **Recommendation** – cold-start user/item recommendation
- **Medical Imaging** – rare disease/organ/tumor segmentation
- **Cybersecurity** – malware family classification with few labeled samples *(directly relevant to our project)*

Few-Shot Medical Image Segmentation: Support image + mask $\rightarrow$ feature extractor $\rightarrow$ organ prototype $\rightarrow$ compare query $\rightarrow$ predicted mask. Evaluated using **Dice** and **IoU**; key challenge is **domain gap** across scanners/hospitals.

Source: Dissanayake et al., 2025

Few-Shot Learning in Our Android Malware Detection Project

Research question: Can a malware family be recognized with only a few labeled examples, and can a completely unseen family be flagged as UNKNOWN rather than misassigned to a known family?



Applying ALP-Net and SSL-ALP-Net Ideas to Malware Classification

- **ALP-Net → Local behavioural prototypes.** Instead of one global average per malware family, compute several prototypes per family (e.g., per permission group, API-call group, or opcode cluster) so variants inside a family are not averaged away. A new APK is matched to the nearest prototype; if it is too far from all of them, it is flagged **UNKNOWN**.
- **SSL-ALP-Net → Self-supervised pretraining on unlabeled APKs.** Cluster large pools of unlabeled APKs into pseudo-families, build support/query episodes from them (with augmentation of the same sample), and pretrain the embedding without analyst labels. Real family labels are then needed only for the few-shot support set.

Caution from the paper: SSL worked for organs but failed for tumors. Likewise, auto-generated pseudo-families may fail for subtle families that do not form clean clusters – this must be validated experimentally.

Expected Insight, Challenges & Future Directions

Expected / target insight:

- Effect of decreasing shots (20-shot down to 1-shot) on accuracy
- Whether the threshold reliably separates known vs. unseen families
- How quickly the system adapts once a new family gets a few labels

Challenges & future directions:

- Choosing the right distance threshold for known/unknown separation
- Domain gap between malware families collected at different times
- Balancing few-shot accuracy against false-unknown / false-known rates
- Extending toward foundation-model-based malware embeddings

Conclusion

- **Few-Shot Learning addresses data scarcity** by enabling models to generalize from a small number of labeled examples using prior knowledge.
- The main challenge is that **limited samples make empirical risk unreliable**, increasing estimation error and making conventional ERM prone to poor generalization.
- **Episodic training and metric-based learning** allow a model to repeatedly learn the few-shot setting. Prototypical Networks provide a simple mechanism to classify queries by comparing their embeddings with class prototypes.
- Different FSL strategies exploit prior knowledge at the **data, feature, task, and multimodal levels**, while applications such as medical imaging (ALP-Net, SSL-ALP-Net) show the importance of adapting methods to the target domain.
- These principles can be extended to **Android malware family recognition**, where the objective is to identify malware families from few labeled samples while detecting genuinely unseen families as **UNKNOWN**.

References

1. Y. Wang, Q. Yao, J. T. Kwok, and L. M. Ni, "Generalizing from a Few Examples: A Survey on Few-Shot Learning," *ACM Computing Surveys*, vol. 53, no. 3, Article 63, 2020.
2. Y. Song, T. Wang, P. Cai, S. K. Mondal, and J. P. Sahoo, "A Comprehensive Survey of Few-Shot Learning: Evolution, Applications, Challenges, and Opportunities," *ACM Computing Surveys*, vol. 55, no. 13s, Article 271, 2023.
3. T. Dissanayake, Y. George, D. Mahapatra, S. Sridharan, C. Fookes, and Z. Ge, "Few-Shot Learning for Medical Image Segmentation: A Review and Comparative Study," *ACM Computing Surveys*, vol. 58, no. 1, Article 11, 2025.

Thank You